



华中科技大学

操作系统原理课程设计报告

姓 名：王家乐

学 院：计算机科学与技术学院

专 业：计算机科学与技术

班 级：计算机本硕博 2301 班

学 号：U202315763

指导教师：廖小飞

分数	
教师签名	

2026 年 3 月 15 日

目 录

实验一 lab1_challenge2 打印异常代码行	1
1.1 实验目的	1
1.2 实验内容与实验记录	1
1.3 实验调试及心得	3
实验二 lab1_challenge3 多核启动及运行	4
2.1 实验目的	4
2.2 实验内容与实验记录	4
2.3 实验调试及心得	7
实验三 lab2_challenge2 堆空间管理	8
3.1 实验目的	8
3.2 实验内容与实验记录	8
3.3 实验调试及心得	11
实验四 lab2_challenge3 多核内存管理	12
4.1 实验目的	12
4.2 实验内容与实验记录	12
4.3 实验调试及心得	15
实验五 lab3_challenge2 实现信号量	15
5.1 实验目的	15
5.2 实验内容与实验记录	15
5.3 实验调试及心得	18
实验六 lab3_challenge3 写时复制	19
6.1 实验目的	19
6.2 实验内容与实验记录	19
6.3 实验调试及心得	22
实验七 lab4_challenge2 重载执行	23
7.1 实验目的	23
7.2 实验内容与实验记录	23
7.3 实验调试及心得	25
实验八 lab4_challenge3 简易 Shell	26
8.1 实验目的	26
8.2 实验内容与实验记录	26
8.3 实验调试及心得	29
challengeX : Put it all together	30

实验一 lab1_challenge2 打印异常代码行

1.1 实验目的

lab1_challenge2 的实验目的是让内核在用户态程序发生运行时异常(如非法指令、访存异常等)时,不再只给出“异常类型”这种粗粒度信息,而是反查出出错位置并打印 file:line 以及对应的源代码行。

1.2 实验内容与实验记录

本实验的核心思路是:在加载用户程序 ELF 文件时,额外读取其中的 DWARF 调试信息 .debug_line 段,利用实验框架中已经提供的 make_addr_line() 函数解析该段内容,建立“指令地址 PC → 文件名/行号”的映射表;当运行时发生异常时,再根据异常现场保存的 mepc 或 sepc 反查对应的源代码位置,并输出异常文件名、行号以及该行源码。

由于 .debug_line 段需要在内核中读入并解析,因此首先需要在 kernel/config.h 中为其预留一块内存区域。我的做法是在原有用户栈、内核栈和 trapframe 之外,新增一块专门用于保存调试信息的空间:

```
#define USER_DEBUG_INFO_BASE 0x82000000
#define USER_DEBUG_INFO_SIZE 0x00200000 // 2MB
```

这里 USER_DEBUG_INFO_BASE 表示调试信息加载到的起始地址, USER_DEBUG_INFO_SIZE 表示最大可用大小。之所以这样设计,是因为在 lab1 中内核虚实地址一致,可以直接给出一段固定地址供 .debug_line 使用。

随后在 kernel/elf.c 中扩展 ELF 加载流程,由于原有加载流程只处理程序段,不能直接按名称访问调试节,因此我先实现了 elf_find_section(), 用于遍历 ELF 的 section header 表,并结合 .shstrtab 字符串表查找指定节(.debug_line)。

```
static int elf_find_section(elf_ctx *ctx, char *secname, elf_sect_header *out) {
    elf_sect_header shstr, sh;
    char shstrtab[8192];
    uint64 shstr_off = ctx->ehdr.shoff + ctx->ehdr.shentsize * ctx->ehdr.shstrndx;
    elf_fpread(ctx, &shstr, sizeof(shstr), shstr_off);
    elf_fpread(ctx, shstrtab, shstr.size, shstr.offset);
    for (int i = 0; i < ctx->ehdr.shnum; i++) {
        uint64 off = ctx->ehdr.shoff + i * ctx->ehdr.shentsize;
        elf_fpread(ctx, &sh, sizeof(sh), off);
        if (strcmp(shstrtab + sh.name, secname) == 0) {
```

```

    *out = sh;
    return 0;
}
}
return -1;
}

```

该函数首先读取 section header string table 对应的节头，再将字符串表内容读入内存，之后遍历所有 section header，通过 sh.name 在字符串表中取得节名，并与目标名称比较，若找到目标节，就返回其节头信息。此基础上，elf_load() 完成后查找 .debug_line 段，并将其读入上述内存区域，再调用 make_addr_line() 解析。核心代码如下：

```

elf_sect_header dbg;
if (elf_find_section(&elfloader, ".debug_line", &dbg) == 0) {
    char *debug_buf = (char *)USER_DEBUG_INFO_BASE;
    if (elf_fpread(&elfloader, debug_buf, dbg.size, dbg.offset) != dbg.size) {
        panic("Fail on loading .debug_line section");
    }
    make_addr_line(&elfloader, debug_buf, dbg.size);
}

```

这样，当前进程中就保存了调试信息对应的目录表、文件表以及地址—行号映射表。接着，我继续在 kernel/elf.c 中实现了 print_errorline(uint64 fault_pc)，其功能是根据异常 PC 在映射表中查找最接近的地址项，得到出错文件和行号，再拼接出源码路径，并从宿主机读取对应代码行打印出来。关键实现如下：

```

void print_errorline(uint64 fault_pc) {
    addr_line *al = lookup_addr_line(current, fault_pc);
    if (!al) return;
    char path[256];
    if (build_path(current, al->file, path, sizeof(path)) != 0) return;
    sprintf("Runtime error at %s:%ld\n", path, (long)al->line);
    // Print the actual source code line.
    char linebuf[256];
    if (read_source_line(path, al->line, linebuf, sizeof(linebuf)) == 0) {
        sprintf(" %s\n", ltrim(linebuf));
    }
}

```

最后，在异常处理路径中调用该函数：对于 kernel/machine/mtrap.c 中的非法指令、访存异常、非对齐访问等异常，在 panic 前加入 print_errorline()。例如：

```

static void handle_illegal_instruction() {
    print_errorline(read_csr(mepc));
    panic("Illegal instruction!");
}

```

实验完成后，运行 `app_errorline` 测试时，系统能够输出如下结果：

```
huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ spike ./obj/riscv-pke ./obj/app_errorline
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: ./obj/app_errorline
Application program entry point (virtual address): 0x0000000081000000
Switch to user mode...
Going to hack the system by running privilege instructions.
Runtime error at user/app_errorline.c:13
    asm volatile("csrw sscratch, 0");
Illegal instruction!
```

本次实验的 `git` 日志如下：

```
huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit b177df37b3613bfc866b1f82257b9c72d36e80aa (HEAD -> lab1_challenge2_errorline)
Author: Nuyoahwjl <3508529368@qq.com>
Date:   Tue Mar 3 22:23:41 2026 +0800

    my work on lab1_challenge2 is done.

commit a9729996d07ff8fe4b724110ac346331159cdbc1
Merge: d47e4b2 1dc7360
Author: Nuyoahwjl <3508529368@qq.com>
Date:   Tue Mar 3 14:36:54 2026 +0800

    continue to work on lab1_challenge2
```

1.3 实验调试及心得

本实验中新增代码的作用主要集中在三个方面。第一，是增加调试信息的存放区域，如果没有这部分空间，内核虽然能加载用户程序代码，却没有地方保存调试信息，自然也无法在运行时使用。第二，是增加 `elf_find_section()` 以及相关加载逻辑，用于在 ELF 的 `section header` 中定位 `.debug_line` 段，并调用 `make_addr_line()` 解析生成地址到行号的映射表。之所以必须新增这部分代码，因为原有 `elf_load()` 只负责程序运行所需的段，而调试段并不参与程序执行，内核默认不会主动读取它。第三，是实现 `print_errorline()` 并在 `mtrap.c` 的异常处理路径中调用它，把已经解析好的调试信息真正用于运行时异常定位。

在调试时，我觉得最容易出问题的地方有两个。一个是 `.debug_line` 的查找和读取过程，如果没有正确处理 `section header` 字符串表，就会导致找不到目标节，最终无法建立映射。另一个是异常地址和源码行的匹配过程，因为异常时拿到的是 PC 值，而不是直接的行号，所以必须在地址映射表中找到“最接近且不超过该地址”的条目，这样才能较准确地还原出错位置。除此之外，源码行内容的打印也依赖于路径拼接是否正确，如果 DWARF 中记录的目录和文件名没有正确组合，就会出现能打印 `file:line` 但读不到源码行的情况。

实验二 lab1_challenge3 多核启动及运行

2.1 实验目的

lab1_challenge3 的实验目的是在前面单核 PKE 内核的基础上，把系统扩展为能够在 Spike 的多 hart 环境下正确完成多核启动、独立运行和协同退出：既要让不同 hart 能分别进入 M/S/U 三种特权级并各自装载、执行对应的用户程序，又要解决多核下共享资源带来的同步与隔离问题。

2.2 实验内容与实验记录

本实验的核心思路：是在原有单核 PKE 内核基础上引入“按 hart 区分与按 hart 隔离”的设计，一方面通过 tp 保存当前 hartid，使内核在 M/S/U 三种特权级下都能识别当前 CPU；另一方面把进程控制块、时钟 tick、栈空间、trapframe 和退出流程都改为面向多核管理，从而实现 hart0 与 hart1 分别装载并运行不同应用，同时通过同步机制保证 HTIF 这类共享资源只初始化一次，最终在所有核都执行结束后再由 hart0 统一关机。

首先，我在机器态入口 _mentry 中把 spike 传入的 hartid 保存到 tp 寄存器，作为后续各阶段识别当前 hart 的依据。

```
.globl _mentry
_mentry:
    csrw mscratch, x0
    # tp is used as hart-local storage in this lab; keep hartid in tp.
    mv tp, a0
```

在进入 _mentry 之前，a0 中已经由 spike 放入当前 hart 的编号，所以把它写入 tp 后，就相当于为每个 hart 建立了一个本地标识。后面在 C 代码中使用 read_tp() 读取它，就可以区分当前执行流属于 hart0 还是 hart1。

在多核启动阶段，实验要求 HTIF 和 Spike 文件接口只能初始化一次，否则两个 hart 同时初始化共享宿主机接口会造成冲突。因此我在 m_start() 中增加了一个全局同步变量 htif_ready，规定只有 hart0 才执行 spike_file_init() 和设备树初始化，其他 hart 必须等待其完成后才能继续打印和访问宿主机接口。关键代码如下：

```
static volatile int htif_ready = 0;
void m_start(uintptr_t hartid, uintptr_t dtb) {
```

```

if (hartid == 0) {
    spike_file_init();
    init_dtb(dtb);
    sprintf("In m_start, hartid:%d\n", hartid);
    asm volatile("fence rw, rw" ::: "memory");
    htif_ready = 1;
} else {
    while (!htif_ready) { asm volatile("nop"); }
    sprintf("In m_start, hartid:%d\n", hartid);
}
}

```

除了 HTIF 同步之外，我还把机器态中断现场 `g_itrframe` 改成了按 `hart` 下标访问，即 `g_itrframe[NCPU]`，避免多个 `hart` 共用一块中断保存区，导致上下文被互相覆盖。

进入监管态之后，每个 `hart` 需要加载不同的用户程序。实验要求 `hart0` 运行 `app0`，`hart1` 运行 `app1`，因此不能再像单核版本那样默认只加载一个程序。我在 `load_bincode_from_host_elf()` 中根据 `tp` 中保存的 `hartid`，从命令行参数数组中选择对应的应用名。核心代码如下：

```

int hartid = (int)read_tp();
const char *app_name = arg_bug_msg.argv[hartid];

```

仅仅让两个 `hart` 加载不同程序还不够，多核环境下还必须保证每个 `hart` 的运行空间彼此独立，否则它们会共用用户栈、内核栈或 `trapframe` 地址，导致切换或陷入时破坏彼此状态。我在 `load_user_program()` 中采用“按 `hart` 分片”的方式，为不同 `hart` 计算不同的地址区域。代码如下：

```

#define USER_ADDR_STRIDE 0x4000000UL
void load_user_program(process *proc) {
    int hartid = (int)read_tp();
    proc->trapframe = (trapframe *) (USER_TRAP_FRAME + (uint64)hartid * USER_ADDR_STRIDE);
    memset(proc->trapframe, 0, sizeof(trapframe));
    proc->kstack = (uint64) (USER_KSTACK + (uint64)hartid * USER_ADDR_STRIDE);
    proc->trapframe->regs.sp = (uint64) (USER_STACK + (uint64)hartid * USER_ADDR_STRIDE);
    proc->trapframe->regs.tp = (uint64)hartid;
    load_bincode_from_host_elf(proc);
}

```

这里保留 `config.h` 中原有的基地址不变，但通过增加一个固定偏移量 `USER_ADDR_STRIDE`，为 `hart1` 分配一套新的用户栈、内核栈和 `trapframe` 空间。这样两个 `hart` 就不会在同一地址上写自己的栈和陷入上下文。

hartid	用户栈	内核栈	trapframe
0	0x81100000	0x81200000	0x81300000
1	0x85100000	0x85200000	0x85300000

进程管理部分，单核版本中只有一个全局 `current` 指针，用来表示“当前正在运行的进程”。这种写法在双核下显然不成立，因为两个 `hart` 会同时运行两个不同的用户程序。如果仍然只保留一个 `current`，一个 `hart` 刚切换进去，另一个 `hart` 就可能把它改掉。因此我把它改成了按核保存的数组：

```
process* current[NCPU] = {0};
void switch_to(process* proc) {
    int hartid = (int)read_tp();
    current[hartid] = proc;
    write_csr(stvec, (uint64)smode_trap_vector);
    proc->trapframe->kernel_sp = proc->kstack;
    proc->trapframe->kernel_trap = (uint64)smode_trap_handler;
    unsigned long x = read_csr(sstatus);
    x &= ~SSTATUS_SPP;
    x |= SSTATUS_SPIE;
    write_csr(sstatus, x);
    write_csr(sepc, proc->trapframe->epc);
    return_to_user(proc->trapframe);
}
```

与之对应，在 `smode_trap_handler()` 中也不再直接使用单个 `current`，而是先根据 `tp` 取出当前 `hart`，再找到对应的进程控制块。保证了每个 `hart` 在发生系统调用或时钟中断时，都只会恢复和切换回自己的进程，而不会误操作另一个 `hart` 的用户上下文。

实验还要求每个 `hart` 的 `tick` 计数也必须是独立的。因此我把单一时钟计数改成了数组 `g_ticks[NCPU]`，并在 `handle_mtimer_trap()` 中按 `hartid` 累加：

```
static uint64 g_ticks[NCPU] = {0};
void handle_mtimer_trap() {
    int hartid = (int)read_tp();
    g_ticks[hartid] += 1;
    write_csr(sip, read_csr(sip) & ~SIP_SSIP);
}
```

最后，最容易忽略但又非常重要的一点，是退出和关机逻辑。在单核版本中，用户程序执行 `exit` 后，内核可以立刻 `shutdown`；但在双核环境中，如果 `hart0` 先退出就立即关机，那么 `hart1` 可能还没有执行完。因此我修改了 `sys_user_exit()`，引入一个全局退出计数 `exited_harts`，只允许 `hart0` 在确认所有 `hart` 都已经退出后，统一执行 `shutdown`。核心实现如下：

```
static volatile int exited_harts = 0;
ssize_t sys_user_exit(uint64 code) {
    int hartid = (int)read_tp();
    sprintf("hartid = %d: User exit with code:%d.\n", hartid, code);
```



```

atomic_add_w(&exited_harts, 1);
if (hartid == 0) {
    while (exited_harts < NCPU) { asm volatile("wfi");}
    sprintf("hartid = %d: shutdown with code:%d.\n", hartid, code);
    shutdown(code);
}
while (1) { asm volatile("wfi"); }
return 0;
}

```

采用原子加法 `amoadd.w` 来更新退出计数，避免两个 `hart` 同时退出时发生竞态。非 0 号 `hart` 在退出后进入 `wfi` 等待，而 `hart0` 则在观察到 `exited_harts = NCPU` 后再执行最终关机。运行 `app0` 和 `app1` 测试，实验结果如下：

```

huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ spike -p2 ./obj/riscv-pke ./obj/app0 ./obj/app1
HTIF is available!
(Emulated) memory size: 2048 MB
In m_start, hartid:0
hartid = 0: Enter supervisor mode...
hartid = 0: Application: ./obj/app0
hartid = 0: Application program entry point (virtual address): 0x0000000081000000
hartid = 0: Switch to user mode...
In m_start, hartid:1
hartid = 1: Enter supervisor mode...
hartid = 1: Application: ./obj/app1
hartid = 1: Application program entry point (virtual address): 0x0000000085000000
hartid = 1: Switch to user mode...
hartid = 0: >>> app0 is expected to be executed by hart0
hartid = 1: >>> app1 is expected to be executed by hart1
hartid = 0: User exit with code:0.
hartid = 1: User exit with code:0.
hartid = 0: shutdown with code:0.
System is shutting down with exit code 0.

```

可见，系统在双核环境下能够正常进入监管态和用户态，`hart0` 与 `hart1` 分别装载并执行各自的用户程序，陷入与时钟中断处理不会互相覆盖，HTIF 只由 `hart0` 初始化一次，最终在两个程序都退出后由 `hart0` 统一关闭模拟器。

本次实验的 `git` 日志如下：

```

huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit 2ed204cc3a5ae42d3ddad24fb50605b28c72f322 (HEAD -> lab1_challenge3_multicore)
Author: Nuyoahwjl <3508529368@qq.com>
Date:   Wed Mar 4 19:53:48 2026 +0800

    my work on lab1_challenge3 is done.

commit b5e332861c19ed1c8442915dbe63e057d11687aa
Merge: 0fa9773 1dc7360
Author: Nuyoahwjl <3508529368@qq.com>
Date:   Tue Mar 3 22:25:47 2026 +0800

    continue to work on lab1_challenge3

```

2.3 实验调试及心得

实验中新增代码的核心作用，都是围绕多核环境下的“身份区分、状态隔离

和共享同步”展开的：例如在入口处把 `hartid` 保存到 `tp`，是为了让内核在 M/S/U 各特权级中始终知道当前代码运行在哪个 `hart` 上；把 `current`、`tick`、中断现场以及用户栈/内核栈/`trapframe` 改为按 `hart` 分开管理，是因为多核会并发执行，如果仍沿用单核时的全局变量和固定地址，不同核就会互相覆盖运行状态；而新增 HTIF 初始化同步和退出计数逻辑，则是因为 HTIF 属于所有核共享的宿主机接口，必须由一个核统一初始化，并且只有当所有 `hart` 都运行结束后，系统才能安全关机。调试过程中我最深的体会是，很多问题表面上表现为“某个 `hart` 卡住”，但根本原因往往是多核共享资源竞争或上下文没有隔离好，这让我更加理解了操作系统中并发控制和核间同步的重要性。

实验三 lab2_challenge2 堆空间管理

3.1 实验目的

lab2_challenge2 的实验目的，是在前面实验已实现按页分配与释放的基础上，把 `better_malloc/better_free` 从“以页为单位”的内存申请，改造成面向进程堆区的动态内存管理机制，即能够在同一页或连续扩展的堆空间中对小块内存进行切分、分配、回收与复用，并通过维护内存控制块和空闲块组织方式，减少内部碎片、提高空间利用率。

3.2 实验内容与实验记录

本实验中，我首先保留了原有 `allocate_page/free_page` 的按页分配机制，同时在 `kernel/syscall.h` 中增加了 `better_malloc/better_free` 对应的新的系统调用号，使用户态的 `better_malloc` 和 `better_free` 可以进入内核中单独实现的堆分配器：

```
// added @lab2_challenge2
#define SYS_user_better_malloc (SYS_user_base + 4)
#define SYS_user_better_free (SYS_user_base + 5)
```

为了让每个用户进程都拥有独立的堆管理状态，我在 `kernel/process.h` 的 `process` 结构体中加入了三个字段，用于描述当前进程的堆链表和堆空间边界：

```
typedef struct process_t {
    ...
    uint64 heap_head;
    uint64 heap_tail;
    uint64 heap_end;
}process;
```

其中，`heap_head` 表示堆中第一个块头的位置，`heap_tail` 表示最后一个块头的位置，`heap_end` 表示当前已经映射到进程地址空间中的 `heap` 末尾地址。随后在 `kernel/kernel.c` 的用户进程加载过程中对这几个字段进行了初始化：

```
proc->heap_head = 0;
proc->heap_tail = 0;
proc->heap_end = USER_FREE_ADDRESS_START;
```

这里 `heap_end` 初值被设置为 `USER_FREE_ADDRESS_START`，即用户堆的起始虚拟地址，使后续的堆扩展和空闲块管理不再是无状态操作，而是能够持续维护进程自己的堆空间。

具体的堆管理实现上，我定义了堆块头 `heap_blk_t`，用于记录每个内存块的元信息，并要求按 16 字节对齐。`size` 记录当前块可供用户使用的 `payload` 大小，`next` 和 `prev` 构成双向链表，`free` 标记该块当前是否空闲。也就是说，真正返回给用户的地址并不是块头地址，而是块头后面的数据区首地址。这种做法相当于在堆空间内部嵌入“内存控制块”，是本实验实现堆管理的关键：

```
typedef struct heap_blk_t {
    uint64 size;
    uint64 next;
    uint64 prev;
    uint64 free;
} heap_blk_t;
```

在此基础上，我实现了堆扩展逻辑。其基本思路是：当现有空闲块无法满足申请需求时，先按页为该进程分配新的物理页并建立映射，再把新增空间整理成新的空闲块。如果堆尾块本身就是空闲块，并且它刚好连接到旧堆末尾，则直接把新页并入该尾块，减少新的块头开销。

```
static void heap_extend(process* proc, uint64 min_bytes) {
    uint64 pages = (min_bytes + PGSIZE - 1) / PGSIZE;
    uint64 old_end = proc->heap_end;
    for (uint64 i = 0; i < pages; i++) {
        heap_map_one_page(proc, proc->heap_end);
        proc->heap_end += PGSIZE;
    }
    if (proc->heap_head == 0) {
        uint64 bva = USER_FREE_ADDRESS_START;
        heap_blk_t* b = heap_hdr(proc, bva);
        b->size = pages * PGSIZE - sizeof(heap_blk_t);
        b->next = b->prev = 0;
        b->free = 1;
        proc->heap_head = proc->heap_tail = bva;
        return;
    }
}
```

有了堆空间后，better_malloc 的核心策略是“首次适配 + 空闲块分裂”。也就是说，从堆首开始顺序寻找第一个满足大小要求的空闲块；如果该块明显大于所需空间，就把它拆成“已分配块 + 剩余空闲块”两部分，否则直接整块分配。这种策略实现简单，而且对本实验给定程序非常有效。

```
static uint64 heap_alloc(process* proc, uint64 nbytes) {
    uint64 need = heap_align_up(nbytes);
    if (proc->heap_head == 0)
        heap_extend(proc, need + sizeof(heap_blk_t));
    uint64 cur_uva = proc->heap_head;
    while (cur_uva) {
        heap_blk_t* cur = heap_hdr(proc, cur_uva);
        if (cur->free && cur->size >= need)
            return heap_alloc_from(proc, cur_uva, need);
        cur_uva = cur->next;
    }
    heap_extend(proc, need + sizeof(heap_blk_t));
    return heap_alloc_from(proc, proc->heap_tail, need);
}
```

与分配相对应，better_free 的重点不是释放整页，而是先把块标记为空闲，再尝试与前后相邻的空闲块合并。这样做可以显著减少外部碎片，同时为下一次分配复用旧空间创造条件。

```
static void heap_free(process* proc, uint64 user_ptr) {
    uint64 hdr_uva = user_ptr - sizeof(heap_blk_t);
    heap_blk_t* b = heap_hdr(proc, hdr_uva);
    b->free = 1;
    if (b->next) { /* 与后继空闲块合并 */ }
    if (b->prev) { /* 与前驱空闲块合并 */ }
}
```

最后，将新实现的堆分配接口接入系统调用分发逻辑中：

```
uint64 sys_user_better_malloc(uint64 nbytes) {
    return heap_alloc(current, nbytes);
}

uint64 sys_user_better_free(uint64 user_ptr) {
    heap_free(current, user_ptr);
    return 0;
}

case SYS_user_better_malloc:
    return sys_user_better_malloc(a1);
case SYS_user_better_free:
    return sys_user_better_free(a1);
```

运行 app_singlepageheap 进行测试，结果如下：

```

huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ spike ./obj/riscv-pke ./obj/app_singlepageheap
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080008000, PKE kernel size: 0x0000000000008000
free physical memory address: [0x0000000080008000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080005000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_singlepageheap
Application program entry point (virtual address): 0x00000000001008a
Switch to user mode...
hello, world!!!
User exit with code:0.
System is shutting down with exit code 0.

```

本次实验的 git 日志如下:

```

huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit f274a0056d0cfdb54ff9db30fdc95811ba483249 (HEAD -> lab2_challenge2_singlepageheap)
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Mar 5 01:01:56 2026 +0800

    my work on lab2_challenge2 is done.

commit f1a432a6bcbe9d7767437a1a05b18480436f86dc
Merge: 8b0f6ef e32e344
Author: Nuyoahwjl <3508529368@qq.com>
Date: Wed Mar 4 19:56:02 2026 +0800

    continue to work on lab2_challenge2

```

3.3 实验调试及心得

在本次堆空间管理实验的调试过程中,我更清楚地理解了“页分配”和“堆分配”在管理方式上的本质区别:前者适合以页为单位进行粗粒度内存映射,后者则必须借助额外的数据结构对小块内存进行组织和复用,因此我新增了进程级的数据结构来描述堆的整体状态,又加入块头结构记录每个内存块的大小、前后关系和空闲标记,并实现了首次适配、块分裂和相邻空闲块合并等逻辑,使 `better_malloc/better_free` 真正具备堆管理能力,满足释放后复用和连续扩展的要求。在调试时,我体会最深的是堆管理不仅是分配一块内存这么简单,更重要的是维护块之间的关系、处理边界与碎片问题,这让我对操作系统中动态内存管理的实现机制有了更具体的认识,同时也感觉这个实验很有启发性,建议在实验说明中进一步强调块分裂、块合并和堆扩展之间的联系,这样更有助于把整个堆管理过程串联起来。

实验四 lab2_challenge3 多核内存管理

4.1 实验目的

lab2_challenge3 的实验目的，是在双核同时运行两个用户进程的场景下，把 lab2_1~2_3 中实现的分页/物理页管理从单核提升为多核，从而保证：每个进程的 naive_malloc 虚拟地址分配在该进程内连续增长、不同 hart 并发分配时不会拿到同一个物理页、以及全局初始化只执行一次且其它核正确同步后再继续。

4.2 实验内容与实验记录

本节不再重复 lab1_challenge3 中已经实现的 hart 保存、双核装载不同程序、HTIF 初始化同步等内容，也不再重复 lab2_challenge2 中已经完成的堆空间按页扩展与用户态分配接口，而是重点解决一个新的问题：当 hart0 与 hart1 同时运行两个用户进程并并发申请内存时，如何保证全局内存管理状态不被破坏，同时又让每个进程维持各自独立的虚拟地址分配序列。

首先需要处理的是 S 态内存管理初始化的一次性问题。在单核版本中，进入 s_start() 后直接执行 pmm_init()、kern_vm_init() 和分页开启即可；但在双核环境下，如果 hart0 和 hart1 同时进入这里，就会重复构造空闲页链表、重复建立内核页表，导致全局状态被覆盖。为此，我在 kernel/kernel.c 中增加了一个全局同步变量 s_mem_init_done，并约定只有 hart0 执行这部分初始化逻辑，hart1 只在等待完成后单独开启自己的分页寄存器：

```
volatile int s_mem_init_done = 0;
int s_start(void) {
    int hartid = (int)read_tp();
    sprint("hartid = %d: Enter supervisor mode...\n", hartid);
    write_csr(satp, 0);
    if (hartid == 0) {
        pmm_init();
        kern_vm_init();
        enable_paging();
        asm volatile("fence rw, rw" ::: "memory");
        s_mem_init_done = 1;
    } else { while (!s_mem_init_done) asm volatile("nop"); enable_paging(); }
    load_user_program(&user_app[hartid]);
    sprint("hartid = %d: Switch to user mode...\n", hartid);
    switch_to(&user_app[hartid]);
    return 0;
}
```

这里最关键的点在于：pmm_init() 和 kern_vm_init() 属于全局共享状态初始化，只做一次；而 satp 是每个 hart 自己的 CSR，因此分页开启 enable_paging()

需要每个 hart 各做一次。这样修改后，内核页表只会创建一次，但两个核都能正确进入分页模式。

第二个关键问题是共享物理页空闲链表的并发安全。在单核版本里，`alloc_page()` 和 `free_page()` 直接对 free list 做头插和弹出即可，不会发生竞争；但本实验中两个 hart 可能同时调用 `naive_malloc`、缺页处理、或者页表构建逻辑，如果不加保护，就有可能出现两个核同时取走同一个空闲页，或者链表头指针被并发修改而损坏。为此，我在 `kernel/pmm.c` 中实现了一个基于 `amoswap.w` 的自旋锁，并将所有 free list 的修改都放到锁的保护范围内。核心代码如下：

```
typedef struct {
    volatile uint32 locked;
} spinlock_t;
static spinlock_t pmm_lock = {0};
static inline void pmm_lock_acquire() {
    uint32 old;
    do {
        asm volatile("amoswap.w.aq %0, %2, (%1)"
            : "=r"(old)
            : "r"(&pmm_lock.locked), "r"(1)
            : "memory");
    } while (old != 0);
}
static inline void pmm_lock_release() {
    asm volatile("amoswap.w.rl x0, %1, (%0)"
        :
        : "r"(&pmm_lock.locked), "r"(0)
        : "memory");
}
```

在此基础上，`free_page()` 改为在插入链表时先上锁、再修改头指针，最后解锁；`alloc_page()` 则在弹出空闲页时也先上锁，确保同一时刻只有一个 hart 能访问共享链表：

```
void free_page(void *pa) {
    pmm_lock_acquire();
    list_node *n = (list_node *)pa;
    n->next = g_free_mem_list.next;
    g_free_mem_list.next = n;
    pmm_lock_release();
}

void *alloc_page(void) {
    pmm_lock_acquire();
    list_node *n = g_free_mem_list.next;
    if (n) g_free_mem_list.next = n->next;
    pmm_lock_release();
    return (void *)n;
}
```


第三个问题是每个进程的简单堆虚拟地址游标必须隔离。在 lab2_challenge2 中，由于系统里只有一个用户进程，naive_malloc 使用单一的全局虚拟地址游标即可；但在本实验中，hart0 和 hart1 分别运行不同进程，如果仍共用同一个游标，就会导致两个进程交替推进地址。为此，我将堆分配起点改成按 hart 维护的数组 g_ufree_page[NCPU]，并在加载用户程序时将当前 hart 对应进程的游标重置为 USER_FREE_ADDRESS_START。在 sys_user_allocate_page() 中，则通过 g_ufree_page[hartid] 为当前进程分配新的虚拟页地址，并在映射完成后将该游标增加一个页大小。相关代码如下：

```
uint64 sys_user_allocate_page() {
    int hartid = (int)read_tp();
    process *p = current[hartid];
    void* pa = alloc_page();
    uint64 va = g_ufree_page[hartid];
    g_ufree_page[hartid] += PGSIZE;
    user_vm_map((pagetable_t)p->pagetable, va, PGSIZE, (uint64)pa,
                prot_to_type(PROT_WRITE | PROT_READ, 1));
    sprintf("hartid = %d: vaddr 0x%x is mapped to paddr 0x%x\n", hartid, (int)va, (uint64)pa);
    return va;
}
```

这样修改后，两个进程都会从 USER_FREE_ADDRESS_START 开始各自分配，因而每个进程内部的虚拟地址保持连续，而不同进程之间互不影响。

运行 app_alloc0、app_alloc1 进行测试，结果如下：

```
hartid = 1: alloc page 0x87f9b000
hartid = 1: vaddr 0x00403000 is mapped to paddr 0x87f9b000
hartid = 0: alloc page 0x87f9a000
hartid = 0: vaddr 0x00404000 is mapped to paddr 0x87f9a000
>>> user alloc 1 @ vaddr 0x00403000
=== user alloc 0 @ vaddr 0x00404000
hartid = 1: alloc page 0x87f99000
hartid = 1: vaddr 0x00404000 is mapped to paddr 0x87f99000
=== user 0: 0
>>> user alloc 1 @ vaddr 0x00404000
=== user 0: 1
>>> user 1: 5
=== user 0: 2
>>> user 1: 6
=== user 0: 3
>>> user 1: 7
=== user 0: 4
>>> user 1: 8
hartid = 0: User exit with code: 0.
>>> user 1: 9
hartid = 1: User exit with code: 0.
hartid = 0: shutdown with code: 0.
System is shutting down with exit code 0.
```

本次实验的 git 日志如下：

```
huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit 2503ea40d19bf6995e11892fae6e3bad358cbe67 (HEAD -> lab2_challenge3_multicoremem)
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Mar 5 10:46:47 2026 +0800

    my work on lab2_challenge3 is done.

commit ab895d02a204b03bc429020d8ff795410891ec49
Merge: 23ac789 e32e344
Author: Nuyoahwjl <3508529368@qq.com>
Date: Wed Mar 4 20:00:15 2026 +0800

    continue to work on lab2_challenge3
```

4.3 实验调试及心得

在本次 lab2_challenge3 多核内存管理实验的调试过程中，我最大的体会是：单核下很多实现，在多核环境里都会因为共享状态而失效，因此调试的关键是要先分清哪些数据是每个进程私有的，哪些数据是全局共享的。例如新增 s_mem_init_done 的作用，是保证 pmm_init() 和 kern_vm_init() 这类全局初始化只执行一次，避免两个核重复建立 free list 和内核页表；新增 pmm_lock 的作用，是保护全局物理页链表，防止两个 hart 并发执行 alloc_page()/free_page() 时拿到同一物理页或破坏链表结构；新增 current[NCPU]、g_ufree_page[NCPU] 和按 hart 设置的 vm_alloc_stage[hartid]，则是为了把“当前进程”“用户堆分配游标”“用户态页分配阶段标记”这些原本单核下的全局状态改成每核独立，从而保证每个进程内部虚拟地址连续、缺页处理和页映射都落到正确的页表上。

实验五 lab3_challenge2 实现信号量

5.1 实验目的

Lab3_challenge2 的实验目的，是在 PKE 操作系统中补全进程同步机制，通过扩展系统调用接口和内核进程管理逻辑，实现用户态可用的信号量原语，使多个并发进程能够基于 PV 操作完成互斥与有序协作。

5.2 实验内容与实验记录

本实验的核心思路是在 PKE 的用户态—系统调用—内核调度这条执行链上补全信号量机制：首先在用户库中提供 sem_new、sem_P、sem_V、sem_free 四个接口，供用户程序直接调用；随后在内核中为这四个接口分配新的系统调用号，并实现一个固定大小的信号量池来保存信号量的值、使用状态以及等待队列；最后把 P/V 操作与进程状态切换、就绪队列和调度器连接起来，使得当 P 操作发

现资源不可用时，当前进程能够阻塞并让出处理器，而当 V 操作释放资源时，又能够唤醒一个等待进程重新参与调度。这样就能让多个进程按照预期顺序协作运行，从而完成进程同步功能。

首先，在 `kernel/syscall.h` 中增加四个新的系统调用号，用于支持信号量的创建、P 操作、V 操作和释放，并在 `user/user_lib.h` 和 `user/user_lib.c` 中补充了用户态接口。将 `sem_new`、`sem_P`、`sem_V`、`sem_free` 封装为用户函数，内部统一通过 `do_user_call()` 触发 `ecall` 进入内核。

```
// added @lab3_challenge2
#define SYS_user_sem_new (SYS_user_base + 6)
#define SYS_user_sem_P   (SYS_user_base + 7)
#define SYS_user_sem_V   (SYS_user_base + 8)
#define SYS_user_sem_free (SYS_user_base + 9)
```

接着我实现了一个固定大小的信号量池，而没有为每个信号量单独动态分配内存，并用 `used` 标志表示该槽位是否已被占用。信号量结构如下：

```
#define NSEM 16
typedef struct semaphore_t {
    int used;
    int value;
    process *wait_head;
    process *wait_tail;
} semaphore;
static semaphore sem_pool[NSEM];
```

这里的 `value` 保存当前信号量值，`wait_head` 和 `wait_tail` 组成一个等待队列，保存由于执行 P 操作而被阻塞的进程。为了避免额外定义新的队列节点结构，我直接复用了进程控制块中的 `queue_next` 指针，让进程既可以进入就绪队列，也可以进入某个信号量的等待队列。随后，我实现了两个辅助函数 `sem_enqueue()` 和 `sem_dequeue()`，分别用于把阻塞进程插入等待队列末尾，以及从队首取出一个等待进程。

在此基础上，我实现了四个系统调用函数。`sys_user_sem_new()` 的逻辑最简单，就是在信号量池中寻找空闲槽位，初始化 `used`、`value` 和等待队列指针，然后返回这个信号量的编号：

```
ssize_t sys_user_sem_new(int init_val) {
    for (int i = 0; i < NSEM; i++) {
        if (!sem_pool[i].used) {
            sem_pool[i].used = 1;
            sem_pool[i].value = init_val;
            sem_pool[i].wait_head = NULL;
            sem_pool[i].wait_tail = NULL;
            return i;
        }
    }
    return -1;
}
```

P 操作是本实验最关键的部分。我采用经典信号量语义：先将 `value` 减 1，

如果结果小于 0，则说明当前没有可用资源，当前进程必须阻塞。阻塞时要做三件事：把当前进程状态改为 BLOCKED，把它插入该信号量的等待队列，然后调用调度器切换到其他可运行进程。代码如下：

```
ssize_t sys_user_sem_P(int semid) {
    if (!sem_valid(semid)) return -1;
    semaphore *sem = &sem_pool[semid];
    sem->value--;
    if (sem->value < 0) {
        // block current process on this semaphore.
        current->status = BLOCKED;
        sem_enqueue(sem, current);
        // switch to another ready process.
        schedule();
    }
    return 0;
}
```

与之对应，V 操作负责释放资源并唤醒等待进程。我的逻辑是先将 value 加 1，如果结果仍然小于等于 0，说明此前一定存在等待者，那么就从等待队列中取出一个进程，并调用 insert_to_ready_queue() 将其放回就绪队列，等待后续调度运行。代码如下：

```
ssize_t sys_user_sem_V(int semid) {
    if (!sem_valid(semid)) return -1;
    semaphore *sem = &sem_pool[semid];
    sem->value++;
    if (sem->value <= 0) {
        process *p = sem_dequeue(sem);
        if (p) { insert_to_ready_queue(p); }
    }
    return 0;
}
```

我还实现了 sys_user_sem_free()，用于释放信号量。为了保证逻辑安全，我规定只有在等待队列为空时才允许释放；如果还有进程阻塞在该信号量上，则返回失败。代码如下：

```
ssize_t sys_user_sem_free(int semid) {
    if (!sem_valid(semid)) return -1;
    semaphore *sem = &sem_pool[semid];
    if (sem->wait_head) return -1;
    sem->used = 0;
    sem->value = 0;
    return 0;
}
```

实现完上述内核函数后，把它们接入系统调用分发函数 do_syscall()。接着结合 app_semaphore.c 对功能进行验证。该程序首先创建一个初值为 1 的 main_sem 和两个初值为 0 的子进程信号量 child_sem[0]、child_sem[1]，然后

通过两次 `fork()` 构造父进程和两个子进程，并用信号量严格控制它们的打印顺序。初始时只有 `main_sem=1`，因此父进程能先通过 `sem_P(main_sem)` 输出；父进程输出后执行 `sem_V(child_sem[0])`，唤醒第一个子进程；子进程 0 输出后再执行 `sem_V(child_sem[1])`，唤醒第二个子进程；子进程 1 输出后执行 `sem_V(main_sem)`，再次唤醒父进程，如此反复。若我的 P/V 实现正确，那么输出顺序就会稳定地按这个循环进行；而如果 P 没有把进程阻塞并调度出去，或者 V 没有正确唤醒等待者，就会出现顺序混乱、卡死或者某个进程一直无法运行的现象。最终测试结果如下：

```
going to insert process 0 to ready queue.
going to schedule process 0 to run.
Parent print 8
going to insert process 1 to ready queue.
going to schedule process 1 to run.
Child0 print 8
going to insert process 2 to ready queue.
going to schedule process 2 to run.
Child1 print 8
going to insert process 0 to ready queue.
going to schedule process 0 to run.
Parent print 9
going to insert process 1 to ready queue.
User exit with code:0.
going to schedule process 1 to run.
Child0 print 9
going to insert process 2 to ready queue.
User exit with code:0.
going to schedule process 2 to run.
Child1 print 9
User exit with code:0.
```

本次实验的 git 日志如下：

```
huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit c879e15ffee46e2938a962906d008615bae79eaa (HEAD -> lab3_challenge2_semaphore)
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Mar 5 11:40:20 2026 +0800

    my work on lab3_challenge2 is done.

commit e8a43a93dc260a8bfec910e731bcdd59f002ad19
Merge: 06f549a 7f1f6a1
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Mar 5 10:51:08 2026 +0800

    continue to work on lab3_challenge2
```

5.3 实验调试及心得

在本次实验中，我在调试过程中进一步理解了信号量的本质并不只是对一个

整数做加减，而是要把 PV 操作和进程状态切换、等待队列管理以及调度器协同起来：sem_new 的新增是为了在内核中为用户进程建立可管理的同步对象，sem_P 的新增是为了在资源不足时把当前进程从运行态转为阻塞态并挂入对应信号量的等待队列，sem_V 的新增则是为了在释放资源后从等待队列中唤醒一个进程重新进入就绪队列，保证并发进程能够继续按照预期次序执行，而 sem_free 则补全了信号量生命周期管理。调试时最关键的地方在于判断条件和队列操作是否正确，例如 P 操作必须在 value < 0 时阻塞、V 操作必须在 value <= 0 时唤醒，否则很容易出现输出顺序错误、进程永久阻塞或无法被重新调度的问题。

实验六 lab3_challenge3 写时复制

6.1 实验目的

lab3_challenge3 的实验目的，是在 PKE 的 fork 机制中实现一种更高效的进程地址空间复制策略：对于父进程堆段，不再在 fork 时立即为子进程分配新物理页并拷贝数据，而是先让父子进程共享同一物理页，只将页表项改为只读并利用 PTE 中未使用的 RSW 位标记该页处于 COW 状态；当子进程后续真正执行写操作、触发缺页异常时，内核再在异常处理中完成实际复制，从而达到“先映射、后复制”的优化效果。

6.2 实验内容与实验记录

在具体实现中，我首先修改了 kernel/riscv.h，利用 RISC-V 页表项中预留给软件使用的 RSW 位定义了 PTE_COW 标记，用来区分普通只读页和“因写时复制而暂时只读”的页。这样在页故障处理时，内核就能准确判断当前异常是否由 COW 引起，而不是把所有写异常都当成非法访问。

```
#define PTE_RSW0 (1L << 8)
#define PTE_RSW1 (1L << 9)
#define PTE_COW PTE_RSW0
```

接着，我在 kernel/pmm.c 和 kernel/pmm.h 中加入了物理页引用计数数组及其操作函数。这里的设计目的，“父进程 fork 多个子进程后，共享页何时才能释放”的问题。新分配出的物理页引用计数初始化为 1；当 fork 让新的子进程共享该页时，引用计数加 1；当 COW 真正发生、旧页不再被当前进程使用，或进程退出解除映射时，引用计数减 1；只有当计数减到 0 时，才调用 free_page() 真正把物理页归还给空闲链表。

```
#define NPAGE ((PHYS_TOP - DRAM_BASE) / PGSIZE)
static uint16 g_page_refcnt[NPAGE];
void page_ref_inc(void* pa) {
    int idx = pa2refidx((uint64)pa);
    if (idx < 0) return;
```

```

    g_page_refcnt[idx]++;
}
int page_ref_dec(void* pa) {
    int idx = pa2refidx((uint64)pa);
    if (idx < 0) return 0;
    assert(g_page_refcnt[idx] > 0);
    g_page_refcnt[idx]--;
    return g_page_refcnt[idx];
}

```

在 kernel/process.c 的 do_fork() 中, 原先的做法是子进程每个堆页都立即复制一份, 开销较大; 现在则我接把子进程对应虚拟地址映射到父进程当前使用的物理页, 然后将父子双方对应页表项的 PTE_W 写权限清除, 并打上 PTE_COW 标记。这样一来, 父子进程在 fork 完成后都只能读不能写, 一旦后续某一方尝试写入, 就会进入页故障处理流程。与此同时, 共享页的引用计数需要增加一次。由于父进程当前仍在运行, 其页表权限发生了变化, 因此最后还需要执行 flush_tlb() 刷新 TLB, 保证新的只读属性能立即生效。

```

// kernel/process.c : do_fork() 中 HEAP_SEGMENT 的处理
case HEAP_SEGMENT:
    ...
    for (uint64 heap_block = parent->user_heap.heap_bottom;
         heap_block < parent->user_heap.heap_top; heap_block += PGSIZE) {
        if (free_block_filter[(heap_block - heap_bottom) / PGSIZE])
            continue;
        uint64 shared_pa = lookup_pa(parent->pagetable, heap_block);
        // map into child as read-only first
        user_vm_map((pagetable_t)child->pagetable, heap_block, PGSIZE, shared_pa,
                     prot_to_type(PROT_READ, 1));
        // mark both mappings as COW and read-only
        pte_t* pte_parent = page_walk(parent->pagetable, heap_block, 0);
        pte_t* pte_child = page_walk(child->pagetable, heap_block, 0);
        *pte_parent = (*pte_parent & ~PTE_W) | PTE_COW;
        *pte_child = (*pte_child & ~PTE_W) | PTE_COW;
        // one more mapping to the same physical page
        page_ref_inc((void*)shared_pa);
    }
}

```

完成共享映射后, 还必须在缺页异常处理函数中补上真正的写时复制过程。为此, 我修改了 kernel/strap.c 中的 handle_user_page_fault(): 当异常类型为 CAUSE_STORE_PAGE_FAULT 时, 先根据故障地址找到对应页表项; 若该页表项有效且带有 PTE_COW 标记, 则说明当前页是一个 COW 共享页。此时如果该物理页的引用计数大于 1, 说明该页仍被其他进程共享, 必须新申请一个物理页, 将旧页内容整页复制到新页, 再把当前进程的页表项改指向新页, 同时恢复 PTE_W 写权限并清除 PTE_COW 标记; 如果引用计数已经等于 1, 则说明该页事实上只剩当前进程独占, 此时无需重新分配物理页, 只要把写权限恢复并去掉 COW 标志即可。最后刷新 TLB, 让处理后的页表立即生效。


```

// kernel/strap.c
if (mcause == CAUSE_STORE_PAGE_FAULT) {
    uint64 va = ROUNDDOWN(stval, PGSIZE);
    pte_t* pte = page_walk(current->pagetable, va, 0);
    if (pte && (*pte & PTE_V) && (*pte & PTE_COW)) {
        uint64 old_pa = PTE2PA(*pte);
        uint64 flags = PTE_FLAGS(*pte);
        // if the page is shared, really copy; otherwise, just make it writable.
        if (page_ref_get((void*)old_pa) > 1) {
            void* new_pa = alloc_page();
            memcpy(new_pa, (void*)old_pa, PGSIZE);
            page_ref_dec((void*)old_pa);
            flags &= ~PTE_COW;
            flags |= (PTE_W | PTE_D);
            *pte = PA2PTE((uint64)new_pa) | flags;
        } else {
            flags &= ~PTE_COW;
            flags |= (PTE_W | PTE_D);
            *pte = PA2PTE(old_pa) | flags;
        }
        flush_tlb();
    }
}
}

```

除了 fork 和页故障处理，页面释放逻辑也必须同步修改，否则进程退出时仍可能错误释放共享页。因此我重写了 kernel/vmm.c 中的 user_vm_unmap()，使其按页遍历待解除映射区间。若参数 free 非 0，则先对该页的引用计数减 1，只有减到 0 时才真正执行 free_page(); 否则只清空页表项。这样就避免了父进程退出时直接把仍被子进程使用的页释放掉的错误。

```

void user_vm_unmap(pagetable_t page_dir, uint64 va, uint64 size, int free) {
    uint64 first = ROUNDDOWN(va, PGSIZE);
    uint64 last = ROUNDDOWN(va + size - 1, PGSIZE);
    for (uint64 a = first; a <= last; a += PGSIZE) {
        pte_t* pte = page_walk(page_dir, a, 0);
        if (pte == 0 || (*pte & PTE_V) == 0) continue;
        if (free) {
            uint64 pa = PTE2PA(*pte);
            // for COW-shared pages, only free when the last mapping disappears.
            if (page_ref_dec((void*)pa) == 0) free_page((void*)pa);
        }
        *pte = 0;
    }
    flush_tlb();
}

```

结合 `app_cow` 进行测试。该程序先在父进程堆上申请一页，输出其物理地址，然后执行 `fork`，在子进程中分别输出写入前后的物理地址。实现成功后，子进程写之前的物理地址应与父进程相同，而在执行写操作触发缺页异常后，写之后的物理地址应变为新的页地址，并且输出中会出现 `handle_page_fault` 相关信息。测试结果如下：

```
Application: ./obj/app_cow
CODE_SEGMENT added at mapped info offset:4
Application program entry point (virtual address): 0x0000000000010078
going to insert process 0 to ready queue.
going to schedule process 0 to run.
the physical address of parent process heap is: 0000000087faf000
User call fork.
will fork a child from parent 0.
in alloc_proc. user frame 0x0000000087fad000, user stack 0x000000007ffff000, user kstack 0x0000000087fac000
do_fork map code segment at pa:0000000087fb2000 of parent to child at va:0000000000010000.
going to insert process 1 to ready queue.
User exit with code:0.
going to schedule process 1 to run.
the physical address of child process heap before copy on write is: 0000000087faf000
handle_page_fault: 0000000000400000
the physical address of child process heap after copy on write is: 0000000087fa0000
User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.
```

本次实验的 git 日志如下：

```
huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit 6b5991858defeb7c91feafbe61e5ac8d85c0cdf1 (HEAD -> lab3_challenge3_cow)
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Mar 5 12:33:51 2026 +0800

    my work on lab3_challenge3 is done.

commit 1053aa3cbcd9ae463549d3c189aa1ef30f51fe30
Merge: 452a824 7f1f6a1
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Mar 5 11:42:41 2026 +0800

    continue to work on lab3_challenge3
```

6.3 实验调试及心得

在本次实验中，我最大的体会是，COW 的关键是要把 `fork`、页表权限控制、缺页异常处理和物理页生命周期管理作为一个完整机制统一考虑：如果只在 `fork` 时简单去掉写权限而不增加 `PTE_COW` 标记，内核就无法区分普通写保护页和 COW 页；如果只实现写故障时分配新页并复制内容，而不维护物理页引用计数，就会在多个进程共享同一页时出现页被提前释放或重复释放的问题。因此：`PTE_COW` 用于标识共享页当前处于写时复制状态，并跟踪共享物理页的使用情况，`do_fork()` 中对堆页改为共享映射并清除写权限，是为了把立即复制优化为延迟复制，而对 `store page fault` 的处理，则真正完成了写时才分配新页的核心逻辑；最后对 `user_vm_unmap()` 的修改保证了进程退出时共享页能够被正确回收。实验中我也体会到，很多内核机制看似只改一个点，实际上往往需要在多个模块之间保持一致性，否则就容易出现隐蔽错误。

实验七 lab4_challenge2 重载执行

7.1 实验目的

lab4_challenge2 的实验目的，是在已经完成文件系统与进程管理基础功能的前提下，通过扩展 PKE 内核和系统调用机制，为用户程序实现 `exec` 功能，即让进程能够以可执行文件路径名为参数，从文件系统中读取新的 ELF 程序，并在当前进程内用该程序重新构造代码段、数据段、用户栈和相关地址空间，从而完成对原进程执行映像的替换。

7.2 实验内容与实验记录

在具体实现中，我首先在 `kernel/syscall.h` 中增加了新的系统调用号 `SYS_user_exec`，在 `user/user_lib.h` 中增加函数声明，并在 `user/user_lib.c` 中实现用户态封装。这样，应用程序中的 `exec("/bin/app_ls")` 最终会通过 `ecall` 进入内核。对应代码如下：

```
// kernel/syscall.h
#define SYS_user_exec (SYS_user_base + 30)

// user/user_lib.h
int exec(const char *pathname);

// user/user_lib.c
int exec(const char *pathname) {
    return do_user_call(SYS_user_exec, (uint64)pathname, 0, 0, 0, 0, 0, 0);
}
```

随后，在 `kernel/syscall.c` 中增加 `sys_user_exec`，先将用户传入的路径参数从用户虚拟地址转换为内核可访问地址，再调用 `do_exec(current, pathpa)` 完成真正的重载执行，同时在系统调用分发表中加入 `SYS_user_exec` 对应分支。这样，`exec` 的调用链路就被完整建立起来。

实验中最关键的部分是 `do_exec` 的实现。我在 `kernel/process.c` 中新增了该函数，其主要做法是先为当前进程准备一套新的用户态执行环境，包括新的页表、`trapframe`、映射信息页和用户栈，然后重新初始化堆空间，再调用新的 ELF 装载函数把目标程序加载到新页表中。加载成功后，再用这些新内容替换当前进程原有的 `pagetable`、`trapframe`、`mapped_info` 和 `user_heap`，这样在 `trap` 返回用户态时，处理器就会按照新的入口地址开始执行新程序。关键代码如下：

```
int do_exec(process *proc, const char *path) {
    process newp;
    memset(&newp, 0, sizeof(newp));
    newp.trapframe = (trapframe *)alloc_page();
    newp.pagetable = (pagetable_t)alloc_page();
    newp.mapped_info = (mapped_region *)alloc_page();
}
```

```

uint64 user_stack_pa = (uint64)alloc_page();
newp.trapframe->regs.sp = USER_STACK_TOP;
user_vm_map(newp.pagetable, USER_STACK_TOP - PGSIZE, PGSIZE,
            user_stack_pa, prot_to_type(PROT_WRITE | PROT_READ, 1));
newp.user_heap.heap_top = USER_FREE_ADDRESS_START;
newp.user_heap.heap_bottom = USER_FREE_ADDRESS_START;
if (load_bincode_from_path(&newp, path) < 0)
    return -1;
proc->pagetable = newp.pagetable;
proc->trapframe = newp.trapframe;
proc->mapped_info = newp.mapped_info;
proc->user_heap = newp.user_heap;
return 0;
}

```

为了通过路径名从文件系统中加载 ELF，我还扩展了 kernel/elf.c 中的程序装载逻辑，新增 load_bincode_from_path()。该函数对路径作区分：如果路径以 / 开头，则通过从 VFS 中读取 ELF 文件；否则仍沿用原来的 spike_file_open 方式读取宿主主机文件。这样既兼容了原先 ./obj/app_exec 的启动方式，/bin/app_exec 和的的执行方式。

```

int load_bincode_from_path(process *p, const char *path) {
    if (path[0] == '/') {
        info.use_vfs = 1;
        info.vf = vfs_open(path, O_RDONLY);
    } else {
        info.use_vfs = 0;
        info.f = spike_file_open(path, O_RDONLY, 0);
    }
    ...
    p->trapframe->epc = elfloader.ehdr.entry;
    return 0;
}

```

最后运行 app_exec 进行测试，成上述修改后，系统运行流程变为：内核首先从/bin/app_exec 装载主程序，进入用户态后，app_exec 调用 exec("/bin/app_ls"), 内核捕获该系统调用，重建当前进程地址空间，再从 /bin/app_ls 读入新的 ELF 文件并设置入口地址，最终在返回用户态时不再回到原先的 app_exec 执行点，而是直接开始执行 app_ls。输出中先后出现 Application: /bin/app_exec 与 Application: /bin/app_ls，并正确列出 /RAMDISK0 目录，说明 exec 已经实现了成功时不返回原程序的语义，也验证了路径装载、系统调用分发和地址空间替换三个环节都已完成。结果如下：

```

Application: ./obj/app_exec
CODE_SEGMENT added at mapped info offset:4
Application program entry point (virtual address): 0x0000000000010078
going to insert process 0 to ready queue.
going to schedule process 0 to run.

===== exec /bin/app_ls in app_exec =====
Application: /bin/app_ls
CODE_SEGMENT added at mapped info offset:4
DATA_SEGMENT added at mapped info offset:5
Application program entry point (virtual address): 0x00000000000100b0
-----
ls "/RAMDISK0":
[name]                [inode_num]
-----
User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.

```

本次实验的 git 日志如下：

```

huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit 712438b45f240682e1b529c2b04abe5071f29256 (HEAD -> lab4_challenge2_exec)
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Mar 5 19:41:48 2026 +0800

    my work on lab4_challenge2 is done.

commit 643df3a9ebdf404a4c33c02482b72648b2922fa2
Merge: 7e87b0a 499b62a
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Mar 5 13:13:53 2026 +0800

    continue to work on lab4_challenge2

```

7.3 实验调试及心得

在完成该实验的过程中，我对 `exec` 的实现原理有了更深的理解：它并不是简单地调用另一个程序，而是在当前进程内部重新建立一套新的用户态执行环境，用新 ELF 替换原有代码段、数据段、栈和相关映射后，再从新的入口地址重新开始执行，因此实验中新增 `SYS_user_exec`、用户态 `exec` 封装、`sys_user_exec`、`do_exec` 以及 `load_bincode_from_path` 等代码都是必不可少的；前两者负责把用户请求传入内核，`sys_user_exec` 负责完成参数地址转换，`do_exec` 负责重建页表、`trapframe`、栈和堆等执行现场，而新增的基于 VFS 的 ELF 装载逻辑则是为了按路径从文件系统读取可执行文件，否则 `/bin/app_ls` 无法被正确找到和加载。调试过程中最需要注意的是：一旦地址空间重建不完整，或者没有正确设置 `trapframe->epc` 和用户栈，系统就无法从新程序入口正常运行；同时，路径参数如果不先从用户虚拟地址转换为内核可访问地址，也会导致内核访

问非法内存。我更加清楚地认识到系统调用、文件系统、进程管理和 ELF 装载之间并不是孤立的模块，而是紧密协作完成一次程序执行切换的整体机制。

实验八 lab4_challenge3 简易 Shell

8.1 实验目的

lab4_challenge3 的实验目的，是在已经完成文件系统、进程创建等基础上，进一步把 fork、exec、wait 与 VFS 文件加载机制真正串联起来，使 PKE 能像一个最简化的操作系统命令解释器那样，从 shellrc 中逐行读取命令，先由 shell 创建子进程，再由子进程通过 exec 按路径重载并执行对应的用户程序，同时把一个字符串参数传递给新程序，父进程则通过 wait 阻塞等待子进程结束，以保证各条命令按顺序执行。

8.2 实验内容与实验记录

首先我补充了用户态到内核态的系统调用通路，在用户库中新增了两个封装函数，分别供父进程等待子进程结束，以及供子进程装载新程序使用：

```
//user/user_lib.c
int wait(int pid) {
    return do_user_call(SYS_user_wait, pid, 0, 0, 0, 0, 0, 0);
}
int exec(const char *pathname, const char *arg) {
    return do_user_call(SYS_user_exec, (uint64)pathname, (uint64)arg, 0, 0, 0, 0, 0);
}
// kernel/syscall.c
ssize_t sys_user_wait(long pid) {
    return do_wait(pid);
}
ssize_t sys_user_exec(char *pathva, char *argva) {
    char *pathpa = (char*)user_va_to_pa((pagetable_t)(current->pagetable), (void*)pathva);
    char *argpa = 0;
    if (argva)
        argpa = (char*)user_va_to_pa((pagetable_t)(current->pagetable), (void*)argva);
    return do_exec(current, pathpa, argpa);
}
```

其中，sys_user_exec() 的关键在于先把用户虚拟地址转换成内核可访问的物理地址，再调用 do_exec()，sys_user_wait() 则直接进入 do_wait()。这样一来，Shell 里的 exec(command, para) 和 wait(pid) 就能够真正落到内核实现上。

接着，为了让父进程能够等到子进程结束，我在内核中实现了 wait 机制，并补充了阻塞队列。do_wait() 的逻辑是不断扫描当前进程的子进程：如果目标子

进程已经变成 ZOMBIE，就回收并返回；如果子进程还没结束，就把父进程挂到阻塞队列并重新调度。核心代码如下：

```
int do_wait(int pid) {
    while (1) {
        int has_child = 0;
        for (int i = 0; i < NPROC; i++) {
            process *cp = &procs[i];
            if (cp->parent != current)
                continue;
            if (pid != -1 && cp->pid != (uint64)pid)
                continue;
            has_child = 1;
            if (cp->status == ZOMBIE) {
                cp->status = FREE;
                return cp->pid;
            }
        }
        if (!has_child)
            return -1;
        insert_to_blocked_queue(current);
        schedule();
    }
}
```

同时，我在 kernal/sched.c 调度器中增加了 blocked_queue_head，并实现了 insert_to_blocked_queue() 和 remove_block_and_insert()。当子进程执行 exit 时，内核会在 sys_user_exit() 中先释放当前进程，再尝试把其父进程从阻塞队列移回就绪队列：

```
ssize_t sys_user_exit(uint64 code) {
    sprint("User exit with code:%d.\n", code);
    free_process(current);
    remove_block_and_insert(current);
    schedule();
    return 0;
}
```

这一保证了 Shell 父进程不会和子进程并发地乱序输出，而是严格遵循子进程结束后父进程继续的顺序。

本实验最关键的部分是 exec 的实现。exec 的不是创建新进程，而是保留当前进程身份，替换当前进程的用户态执行映像，这部分的实现同 lab4_challenge2。但是 exec 不仅要替换 ELF，还要考虑参数传递问题。因此，我在 do_exec() 中额外分配了一页参数页，把字符串参数写进去，并手动设置 a0、a1 两个寄存器，让新程序的 main(argc, argv) 能够拿到参数：

```
// 传一个参数给新程序: argc=1, argv[0]=arg
if (arg != NULL) {
    uint64 arg_page_va = USER_FREE_ADDRESS_START;
```

```

uint64 arg_page_pa = (uint64)alloc_page();
memset((void *)arg_page_pa, 0, PGSIZE);
user_vm_map((pagetable_t)newp.pagetable,
            arg_page_va,
            PGSIZE,
            arg_page_pa,
            prot_to_type(PROT_WRITE | PROT_READ, 1));
char *arg_str_va = (char *)arg_page_va;
uint64 *argv_va = (uint64 *) (arg_page_va + 128);
char *arg_page_kva =
    (char *)lookup_pa((pagetable_t)newp.pagetable, arg_page_va);
strcpy(arg_page_kva, karg);
((uint64 *) (arg_page_kva + 128))[0] = (uint64)arg_str_va;
((uint64 *) (arg_page_kva + 128))[1] = 0;
newp.trapframe->regs.a0 = 1; // argc
newp.trapframe->regs.a1 = (uint64)argv_va; // argv
}

```

完成内核支持后，给定的 Shell 程序本身就可以工作了。app_shell.c 的逻辑比较直接：先打开 /shellrc，再按命令和参数解析，每读到一条命令就 fork 一个子进程，子进程调用 exec(command, para)，父进程调用 wait(pid)，等子进程执行结束后继续处理下一条命令，依次执行 hostfs_root/shellrc 中的脚本，结果如下：

```

/bin/app_mkdir /RAMDISK0/sub_dir
/bin/app_touch /RAMDISK0/sub_dir/ramfile1
/bin/app_touch /RAMDISK0/sub_dir/ramfile2
/bin/app_echo /RAMDISK0/sub_dir/ramfile1
/bin/app_cat /RAMDISK0/sub_dir/ramfile1
/bin/app_ls /RAMDISK0/sub_dir
/bin/app_ls /RAMDISK0
END END

```

```

===== mkdir command =====
mkdir: /RAMDISK0/sub_dir
User exit with code:0.
going to insert process 0 to ready queue.
going to schedule process 0 to run.
=====Command End=====

```

```

===== touch command =====
touch: /RAMDISK0/sub_dir/ramfile1
file descriptor fd: 0
User exit with code:0.
going to insert process 0 to ready queue.
going to schedule process 0 to run.
=====Command End=====

```

```

===== touch command =====
touch: /RAMDISK0/sub_dir/ramfile2
file descriptor fd: 0
User exit with code:0.
going to insert process 0 to ready queue.
going to schedule process 0 to run.
=====Command End=====

```

```

===== echo command =====
echo: /RAMDISK0/sub_dir/ramfile1
file descriptor fd: 0
write content:
hello world
User exit with code:0.
going to insert process 0 to ready queue.
going to schedule process 0 to run.
=====Command End=====

```

```

===== cat command =====
cat: /RAMDISK0/sub_dir/ramfile1
file descriptor fd: 0
read content:
hello world
User exit with code:0.
going to insert process 0 to ready queue.
going to schedule process 0 to run.
=====Command End=====

```

```

----- ls command -----
ls "/RAMDISK0/sub_dir":
[name]                [inode_num]
ramfile1               2
ramfile2              3
-----
User exit with code:0.
going to insert process 0 to ready queue.
going to schedule process 0 to run.
=====Command End=====

```

```

----- ls command -----
ls "/RAMDISK0":
[name]                [inode_num]
sub_dir               1
-----
User exit with code:0.
going to insert process 0 to ready queue.
going to schedule process 0 to run.
=====Command End=====

```

本次实验的 git 日志如下：

```

huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit 1530926565c242acd12dba61296fad4e64ee5cf5 (HEAD -> lab4_challenge3_shell)
Author: Nuyoahwjl <3508529368@qq.com>
Date:   Fri Mar 6 15:36:00 2026 +0800

    my work on lab4_challenge3 is done.

commit 344f56f86254e96d2a67261c5c2a9421be2c3629
Merge: 735dbaa 499b62a
Author: Nuyoahwjl <3508529368@qq.com>
Date:   Thu Mar 5 20:15:15 2026 +0800

    continue to work on lab4_challenge3

```

8.3 实验调试及心得

在本次实验的调试过程中，结合实验原理来看，Shell 的本质流程是“读取命令、fork 子进程、子进程 exec 新程序、父进程 wait 等待子进程结束”，因此新增 SYS_user_exec、SYS_user_wait 及用户库中的 exec()、wait() 封装，是为了

让用户态命令控制真正能够下沉到内核；新增 `do_exec()` 的作用是保留原进程的 `pid`、父子关系和文件管理结构，同时重建页表、用户栈、`trapframe` 和映射信息，再从 VFS 按路径装载 ELF；新增 `load_bincode_from_path()` 则是为了让可执行文件通过路径加载；新增阻塞队列、`do_wait()` 以及退出时对父进程的唤醒逻辑，则是为了解决父进程和子进程乱序执行的问题，保证脚本中的多条命令严格串行。调试时我也更深刻地理解了 `argc/argv` 的传递并不是自动完成的，而是需要内核显式准备参数页并设置寄存器 `a0/a1`，这让我把系统调用、地址空间、进程调度和文件系统这几个看似分散的知识点真正联系了起来。

challengeX : Put it all together

`challengeX` 是把前面几个 `challenge` 已经实现的能力真正串起来：一方面在内核侧补齐 `exec` 参数传递、相对路径解析、`wait/waitpid`、多页 ELF 装载等基础支撑，使 `shell` 能像一个统一入口一样调起不同用户程序；另一方面在用户态把原先只能顺序读取 `shellrc` 的简易 `shell`，扩展成可交互、可维护后台任务、可查看历史命令的小型命令解释器。这样，前面分散完成的文件系统、进程管理和程序装载功能就被整合成了一个可以直接操作和验证的实验系统。

本实验是在 `lab4_challenge3_shell` 的基础上继续推进的。首先，我沿用了 `challenge3` 中 `shell` 调用程序的基本框架，但很快发现原有实现只能读取 `shellrc` 并以命令 + 单参数的方式顺序执行，无法支撑 `challengeX` 整合所有 `challenge app` 的要求。因此第一步先从构建系统入手，把前面实验产生的用户程序全部加入编译流程，并统一放到 `hostfs_root/bin` 目录下。这样 `shell` 就不再依赖手工替换单个用户程序，而是可以像普通系统一样通过路径加载不同 `app`。相关修改体现在 `Makefile` 中。

```
USER_APPS := \
    app_shell app_ls app_mkdir app_touch app_cat app_echo \
    app_print_backtrace app_errorline app0 app1 app_sum_sequence \
    app_singlepageheap app_alloc0 app_alloc1 app_wait \
    app_semaphore app_cow app_relativepath app_exec
USER_TARGETS := $(addprefix $(HOSTFS_ROOT)/bin/, $(USER_APPS))
run: $(KERNEL_TARGET) $(USER_TARGETS)
    spike $(KERNEL_TARGET) /bin/app_shell
```

完成这一部分后，`shell` 具备了统一调度多个 `app` 的条件。为了进一步验证所有 `challenge` 的功能，我又配置了 `hostfs_root/shellrc`，让 `shell` 按顺序执行目录创建、文件写入、目录遍历、回溯打印、内存分配、等待、信号量、写时复制、相对路径和 `exec` 等应用。

```
/bin/app_mkdir /RAMDISK0/sub_dir
/bin/app_touch /RAMDISK0/sub_dir/ramfile1
/bin/app_touch /RAMDISK0/sub_dir/ramfile2
/bin/app_echo /RAMDISK0/sub_dir/ramfile1
/bin/app_cat /RAMDISK0/sub_dir/ramfile1
/bin/app_ls /RAMDISK0/sub_dir
/bin/app_ls /RAMDISK0
```

```

/bin/app_print_backtrace
/bin/app0
/bin/app1
/bin/app_singlepageheap
/bin/app_alloc0
/bin/app_alloc1
/bin/app_wait
/bin/app_semaphore
/bin/app_cow
/bin/app_relativepath
/bin/app_exec
# /bin/app_sum_sequence
# /bin/app_errorline
# END END

```

我重点修改了 kernel/process.c 与 kernel/syscall.c，把原先较简单的执行流程补充为完整的路径解析 + 新地址空间建立 + 参数传递过程。这里最关键的一步，是新增 setup_user_args，将参数字符串和 argv[] 压入新程序的用户栈，并通过寄存器 a0/a1 传递给入口函数。这样，shell 启动的用户程序就不再只是“被执行”，而是能像正常命令行程序一样接收参数。

```

int setup_user_args(process *proc, int argc, char **argv) {
    uint64 stack_base = USER_STACK_TOP - PGSIZE;
    char *stack_pa = (char *)lookup_pa((pagetable_t)proc->pagetable, stack_base);
    uint64 sp = USER_STACK_TOP;
    uint64 argv_user[EXEC_MAX_ARGS];
    for (int i = argc - 1; i >= 0; i--) {
        int len = (int)strlen(argv[i]) + 1;
        sp -= len;
        memcpy(stack_pa + (sp - stack_base), argv[i], len);
        argv_user[i] = sp;
    }
    sp &= ~0xFUL;
    sp -= (argc + 1) * sizeof(uint64);
    uint64 *argv_pa = (uint64 *)(stack_pa + (sp - stack_base));
    for (int i = 0; i < argc; i++) argv_pa[i] = argv_user[i];
    argv_pa[argc] = 0;
    proc->trapframe->regs.a0 = argc;
    proc->trapframe->regs.a1 = sp;
    proc->trapframe->regs.sp = sp & ~0xFUL;
    return 0;
}

```

对应地，系统调用层也需要从旧进程的用户空间中读取参数，再把它们交给 do_exec 处理。因此我在 sys_user_exec 中加入了参数数组拷贝逻辑，并将用户态接口扩展为更完整的形式。相关代码如下：

```

ssize_t sys_user_exec(char *pathva, int argc, char **argvva) {

```

```

char *pathpa = (char *)user_va_to_pa((pagetable_t)(current->pagetable), (void *)pathva);
char kpath[MAX_PATH_LEN];
char kargs[EXEC_MAX_ARGS][MAX_PATH_LEN];
char *kargv[EXEC_MAX_ARGS];
resolve_user_path(pathpa, kpath);
for (int i = 0; i < argc; i++) {
    uint64 arg_uva = read_user_word((uint64)argvva + i * sizeof(uint64));
    char *arg_pa =
        (char *)user_va_to_pa((pagetable_t)(current->pagetable), (void *)arg_uva);
    strcpy(kargs[i], arg_pa);
    kargv[i] = kargs[i];
}
return do_exec(current, kpath, argc, argc > 0 ? kargv : 0);
}

```

除了 exec, challengeX 还要求将相对路径 challenge 的成果真正融入 shell 环境, 所以我在 kernel/syscall.c 中统一加入了 resolve_user_path, 让 open、mkdir、opendir、link、unlink、exec 等系统调用都先根据当前工作目录解析路径, 再进入 VFS 层。这使得系统不再只支持绝对路径, 而能够正确处理 ./ 和当前目录相关操作。相关逻辑如下:

```

static void resolve_user_path(const char *user_path, char *out) {
    if (user_path[0] == '/') {
        strcpy(out, user_path);
        return;
    }
    struct dentry *cwd = current->pfiles->cwd;
    const char *cwd_name = cwd ? cwd->name : "/";
    if (user_path[0] == '.' && user_path[1] == '/') {
        out[0] = '/';
        if (cwd_name && cwd_name[0] != '/' && cwd_name[0] != '\0') {
            strcat(out, cwd_name);
            strcat(out, "/");
        }
        strcat(out, user_path + 2);
    }
}

```

接着, 我进一步把基础版顺序 shell 改造成了交互式 shell, 并实现了后台任务管理。这里最关键的新增内核支持是 waitpid(pid, nohang) 与 getchar(): 前者允许 shell 非阻塞地回收后台进程, 后者允许直接从控制台读取用户输入, 从而真正形成 REPL 式命令行。

```

ssize_t sys_user_waitpid(long pid, int nohang) {
    return do_waitpid((int)pid, nohang);
}

ssize_t sys_user_getchar(void) {
    int ch = -1;

```



```

while (ch < 0) ch = htif_console_getchar();
if (ch == '\r') ch = '\n';
return ch;
}

```

对应地，用户态 shell 增加了命令解析、作业表和后台执行逻辑：

```

static int parse_line(char *line, char *argv[], int *background) {
    ...
    if (argc > 0 && strcmp(argv[argc - 1], "&") == 0) {
        *background = 1;
        argc--;
    }
    return argc;
}

static void reap_background_jobs(void) {
    while (1) {
        int pid = waitpid(-1, 1);
        if (pid <= 0) break;
        finish_job(pid, 1);
    }
}

```

这样，shell 已经支持 pwd、cd、history、jobs、waitall、source 等内建命令，并能通过别名把 ls、mkdir、cat、app0 等程序作为外部命令启动。别名如下：

```

static const command_alias g_aliases[] = {
    {"ls", "/bin/app_ls"},
    {"mkdir", "/bin/app_mkdir"},
    {"touch", "/bin/app_touch"},
    {"cat", "/bin/app_cat"},
    {"writehello", "/bin/app_echo"},
    {"backtrace", "/bin/app_print_backtrace"},
    {"errorline", "/bin/app_errorline"},
    {"heap", "/bin/app_singlepageheap"},
    {"alloc0", "/bin/app_alloc0"},
    {"alloc1", "/bin/app_alloc1"},
    {"waitdemo", "/bin/app_wait"},
    {"sem", "/bin/app_semaphore"},
    {"cow", "/bin/app_cow"},
    {"relative", "/bin/app_relativepath"},
    {"execdemo", "/bin/app_exec"},
    {"app0", "/bin/app0"},
    {"app1", "/bin/app1"},
    {"sum", "/bin/app_sum_sequence"},
    {0, 0},
};

```

这部分完成的重点主要是后台执行与 shell 交互能力扩展：命令末尾

加 & 后，进程不再阻塞前台 shell，而是进入作业表，随后由 jobs 查看状态、由 waitall 统一回收；pwd 支持查看当前目录；cd 支持切换目录；history 用来查看历史交互信息；source 命令则用来执行脚本文件。也就是说，所有 challenge app 既支持通过别名调用，也支持通过完整命令调用，也可以将命令写入 shellrc 脚本文件通过运行脚本来执行。最终交互如下：

```

===== PKE Interactive Shell =====
type 'help' to list commands
[pke:/]$ help
Ticks 0
help
=====
|                                     |
|                               PKE Shell Help                               |
|                                     |
|-----|
| Builtins   : help pwd cd history jobs wait waitall                      |
|              source exit                                                  |
|-----|
| Aliases    : ls mkdir touch cat writehello backtrace                    |
|              errorline heap alloc0 alloc1 waitdemo sem                  |
|              cow relative execdemo app0 app1 sum                        |
|-----|
| Background : append '&' at end of command                              |
|-----|
=====

```

所有命令都能正常，且执行 source /shellrc 后，所有 challenge 都可被调用。
本次实验的 git 日志如下：

```

huster@huster-VMware-Virtual-Platform:~/桌面/05/riscv-pke$ git log
commit 2a3a670e8fdd0bf04bc3db80d67ecc6cb2edbd81 (HEAD -> challengeX_bonus)
Author: Nuyoahwjl <3508529368@qq.com>
Date:   Wed Mar 11 19:14:17 2026 +0800

    challengeX加分内容

commit d91d5ef699abf30c38ab9b012333abd46b5e1b94 (challengeX)
Author: Nuyoahwjl <3508529368@qq.com>
Date:   Tue Mar 10 21:01:37 2026 +0800

    challengeX基础内容

commit 1530926565c242acd12dba61296fad4e64ee5cf5 (lab4_challenge3_shell)
Author: Nuyoahwjl <3508529368@qq.com>
Date:   Fri Mar 6 15:36:00 2026 +0800

    my work on lab4_challenge3 is done.

```